

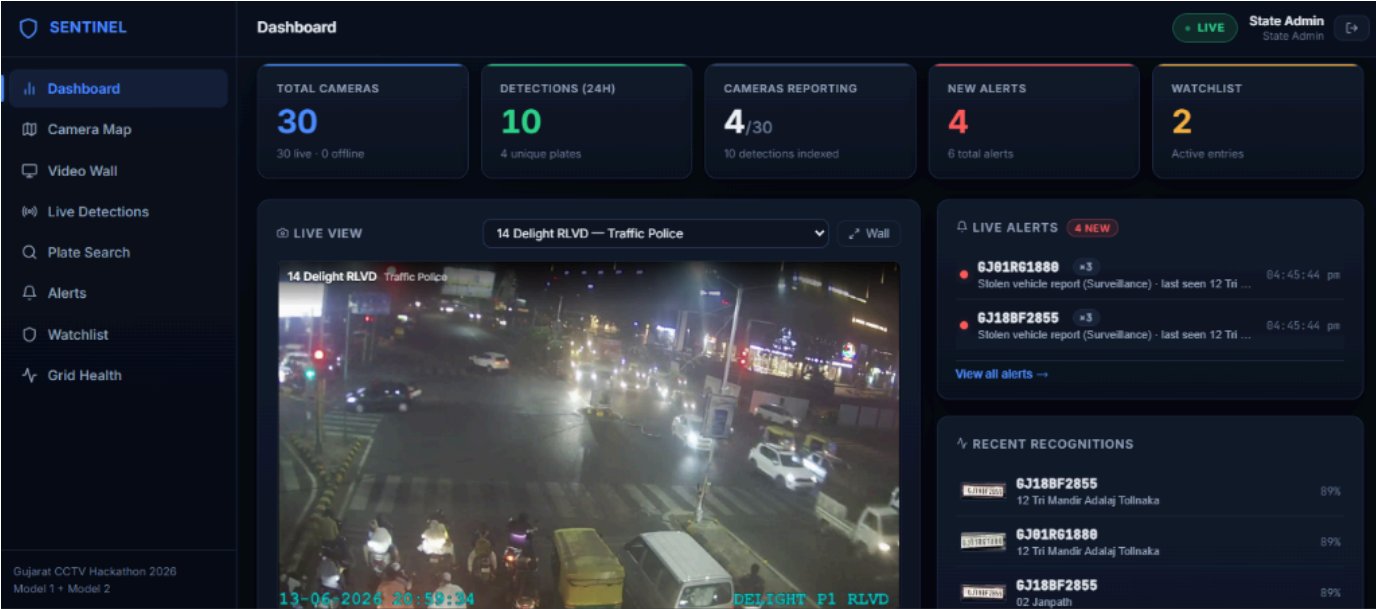
# SENTINEL

## Statewide CCTV Integration Platform & ANPR Surveillance Engine

- **Event:** Gujarat CCTV Integration Hackathon 2026
- **Category:** Category 1 (Academic / Research / Startup)
- **Team Name:** KodeMatrix
- **Team Members:** Devam Shah, Krishit Shah, Moksheet Shah, Abhishek Shah, Yash Rathod
- **Track Scope:** Model 1 (Central CCTV Registry & GIS Mapping) + Model 2 (Unified Viewing Platform & ANPR Alerting)

A production-grade platform running against the live Sentinel sandbox grid: 30 cameras onboarded and mapped, a continuous ANPR pipeline validated at 100% on legible footage, real-time watchlist alerting, and timestamped route reconstruction across cameras.

### Platform Running - Live Dashboard:



## Slide 2a: The Problem

### The Operational Gap in Statewide Surveillance

- Cameras are owned by multiple departments in diverse formats with no central registry.
- Investigators asking "where has this vehicle been?" have no unified system to query across jurisdictions.
- Watchlist matching, where present, relies on manual human monitoring across isolated displays.

## Slide 2b: What Sentinel Delivers

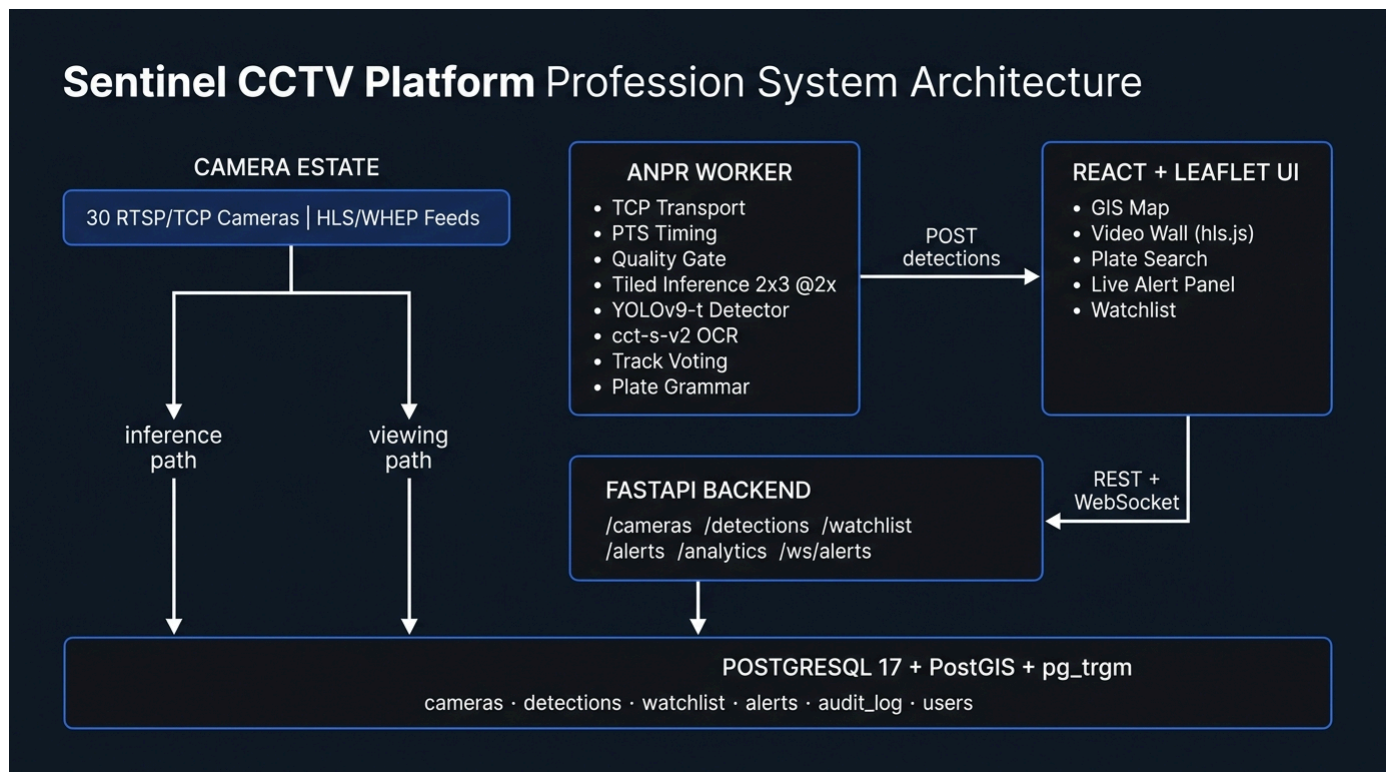
### One platform. Every camera. Automatic.

- **Central Onboarding** - any catalogue camera, location provenance recorded
- **Unified Viewing** - live multi-camera wall, no per-department software
- **Continuous ANPR** - every plate, every camera, every timestamp
- **Watchlist Matching** - exact-then-fuzzy trigram on every detection

- **Real-Time Alerts** - WebSocket dispatch, severity levels, instant
- **Route Reconstruction** - interactive map timeline with speed estimation
- **Complete Auditability** - purpose-bound audit on every search and export

**Alert severity color key:** Red = exact watchlist match | Amber = fuzzy match (score > 0.7) | Blue = informational. All severity levels are also labelled in text for colorblind-safe operation.

## Slide 3: System Architecture



**Technology Stack:** Python 3.12+ / FastAPI | PostgreSQL 17 + PostGIS + pg\_trgm | React + Vite + TypeScript | Leaflet | hls.js | ONNX Runtime (CPU)

## Slide 4: Model 1 - Registry & GIS Mapping

### Handling Real-World Data Gaps with Integrity

The sandbox catalogue publishes **only id and name**: no coordinates, no department, and no codec hints.

### Our Solution

- Resolved all 30 camera locations from real site names published in the catalogue.
- Implemented strict location provenance precedence: hand-verified → Nominatim geocoding (rate-limited, cached, bounded to Gujarat) → district centroid → **null**.
- Stored `geo_source` and `geo_confidence` on every camera record so provenance travels into reports and UI displays.

### What We Refused To Do

An early version placed Junagadh cameras in Surat. We caught it, and removed it.

**GIS Capabilities:** PostGIS geography points, GeoJSON export, department / status / confidence map layers, clustered markers, and click-through to live video tiles.

## Slide 5: Model 2 - ANPR & Accuracy Engine

### Local CPU Inference | Zero Third-Party API Dependence

Pretrained open-source models (YOLOv9-t detector, cct-s-v2 OCR) executing on **local CPU**. No video frame ever leaves the deployment; no external service sits in the alerting path.

### Accuracy Achieved Through Domain-Specific Engineering

- 1. Track-Level Voting:** Vehicles remain visible across 20-60 frames. Every read votes per character weighted by OCR confidence, right-aligned on the 4-digit serial. Tracks reach 20-35 reads and accumulate 1.000 confidence.
- 2. Indian Plate Grammar:** Position rules make common OCR confusions deterministically correctable.

**Real Example:** OCR returned GJ01AB1234 . Position 2 must be a digit - O becomes o unambiguously. 7/7 pass rate on correction test suites. The same logic corrects five other common OCR confusions.

- 3. Overlay Rejection:** Detector filters out burnt-in timestamps, PTZ text overlays, and billboard signage before indexing.

### Real Plate Finding on Live Grid (cam12 Adalaj Toll Plaza):



THE PLATE, MAGNIFIED 9x



A human reads:

**GJ02XX4499**

The recogniser returned:

**66Q2XT449**

state code lost, one digit dropped

After grammar correction:

**GG02X7449**

well-formed, but GG is not a real RTO code - REJECTED

58 px / 10 characters = 5.8 px per character

**The limit is optics, not software.**

Same pipeline reads 6/6 on legible footage. Seven preprocessing variants tried - upscaling cannot restore detail the sensor never captured.

**Empirical Accuracy & Siting Findings:** - **Legible Ground-Truth Feeds ( $\geq 15$  px/char): 6/6 (100%)** complete plate registrations recovered and verified. - **Live Toll Plaza (cam12, 5.8 px/char): 25 live detections** of a genuine truck plate, recovering 8/10 characters ( 66Q2XT449 ). - **Optics vs Software:** Upscaling cannot recreate details the sensor never captured. Siting cameras at toll plazas, checkpoints, and queue lines produces readable plates immediately. Full detail in HLD §6.5.

## Slide 6: Continuous ANPR Engine & Scalability

### Inference Costs & Siting Realities

At 1920x1080 resolution, wide-area night feeds render number plates at 5-15 px wide. Full-frame inference at 384 px proposes zero candidates. Tiled inference on overlapping upscaled regions recovers plate regions at 15x CPU cost.

| Benchmark on cam05 (20-core CPU, No GPU) | Full Frame    | Tiled 2x3 @2x         |
|------------------------------------------|---------------|-----------------------|
| Mean Inference Time                      | 12.3 ms/frame | <b>185.9 ms/frame</b> |
| Est. Concurrent Streams per Machine      | ~251          | <b>~26</b>            |
| Plate Candidates (90s window)            | <b>0</b>      | <b>8</b>              |
| Ground-Truth Plates Recovered            | 3/6           | <b>6/6</b>            |

### Analytics Policy Tiers

ANPR is not a uniform per-camera cost. It is a **policy choice regarding which cameras require analytics:**

| Analytics Tier          | Share of Estate | Deployment Workload                                |
|-------------------------|-----------------|----------------------------------------------------|
| Tier 1: Continuous ANPR | 5-10%           | Highways, border posts, ANPR-sited toll plazas     |
| Tier 2: Event-Triggered | 20-30%          | Urban junctions (motion / signal triggered)        |
| Tier 3: Registry & View | 60-75%          | Full video coverage without analytics compute cost |

## Slide 7: Correlation, Alerting & Route Reconstruction

### Watchlist Matching: Exact + Fuzzy Trigram Search

Every detection is matched against active watchlists before database transaction commit: exact match first, followed by `pg_trgm` trigram similarity > 0.7. Matches generate instant WebSocket alerts with severity metadata.

**Why Fuzzy Search is Critical:** ANPR OCR will occasionally misread a character. Exact-only matching misses genuine detections. Verified: searching `GJ01AB1234` correctly retrieves records stored as `GJ01AB1234` .

### Route Reconstruction

Sightings ordered chronologically - great-circle distance & elapsed time calculated - implied speed estimated - **physically impossible transitions ( $>150$  km/h) flagged** and surfaced to operators rather than hidden. Flagged transitions notify investigators of potential plate misreads or cloned registration plates, providing actionable intelligence.

---

## Slide 8: Security, Privacy & Accountability

---

JWT with three roles, enforced as a dependency on each route - verified: a viewer token returns 403 on the audit trail and 200 on the camera registry.

- **Access Control:** state\_admin / dept\_operator / viewer roles on every protected endpoint
- **Audit Trail:** actor from verified token, purpose and case reference on every search and export
- **Data Minimisation:** plate text, timestamp, camera, crop only - no continuous video retained
- **Data Residency:** 100% local inference, zero frames leave the deployment
- **DPDP Act 2023:** purpose-bound access, role restrictions, complete access trail

### Closed Government Systems (VAHAN, SARTHI, eGujCop)

Contract-first adapter pattern: request/response models defined, mock adapters with realistic records, exact activation steps documented for when production credentials arrive.

---

## Slide 9: Scaling to 80,000 Cameras

---

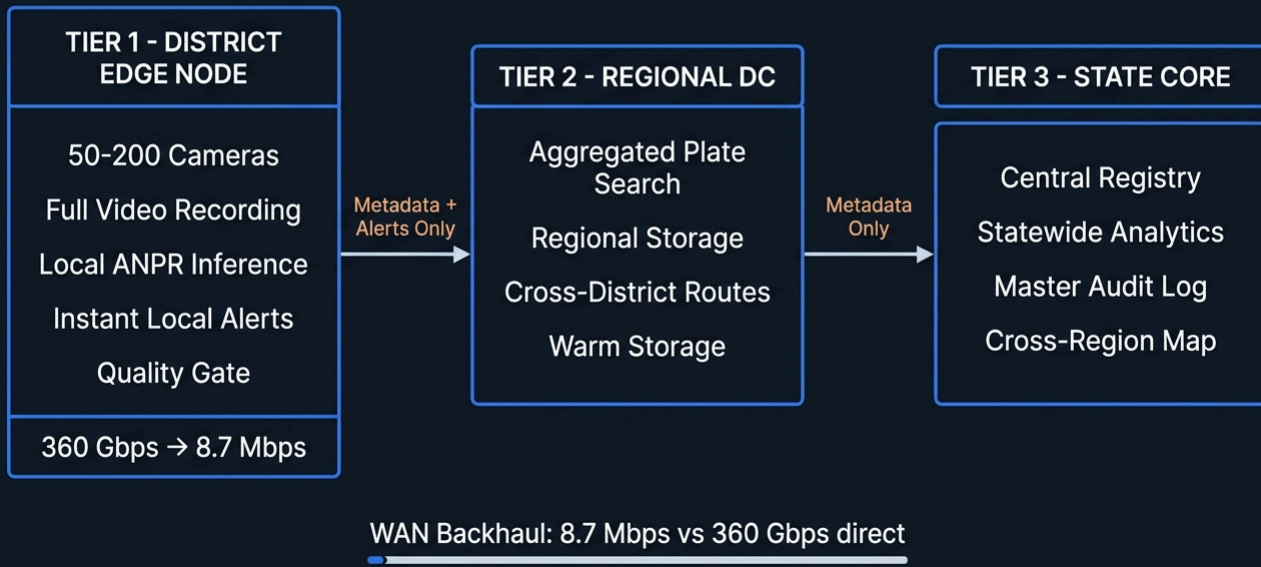
### Measured Bottleneck (not projected)

| Concurrent Streams | Succeeded | Wall Time | Our Server CPU (20 Cores) |
|--------------------|-----------|-----------|---------------------------|
| 2                  | 2 / 2     | 12 s      | 4%                        |
| 4                  | 4 / 4     | 26 s      | 4%                        |
| 6                  | 6 / 6     | 41 s      | 5%                        |
| 8                  | 7 / 8     | 88 s      | 4%                        |

**Key Insight:** Our server sat at 4% CPU while the gateway took 73s to accept 8 connections. The bottleneck is centralized ingress - not compute. This is the 80,000-camera problem at a scale of eight.

### Statewide Edge-First Architecture

# Statewide Edge-First Architecture



Restricting WAN backhaul to metadata, alerts, and requested clips reduces wide-area bandwidth demand from **360 Gbps to 8.7 Mbps** - two to three orders of magnitude.

## Slide 10: Platform Capabilities & Transparent Evaluation

### Fully Demonstrated Capabilities

- 30 sandbox cameras onboarded with full location provenance
- Interactive GIS map with department, status, and confidence filters
- Unified video wall featuring HLS grid and WebRTC hero tiles
- Continuous ANPR pipeline with tiled inference, track voting, and grammar correction
- Exact, partial, and fuzzy plate search
- Automatic watchlist matching with real-time WebSocket alerts
- Route reconstruction with speed calculation and impossible transition flagging
- Output report generation in XLSX and PDF formats
- Purpose-bound audit trail logging
- 57 automated tests, 100% accuracy on legible ground-truth feeds

### Production Safety Architecture: The cam10 Finding

## cam10 Char Chowk Road, Junagadh

The failure mode that matters: well-formed, confident, and wrong

THE EVIDENCE CROP, MAGNIFIED



True plate

**GJ03HR4879**

What the pipeline read

**GJ038988**

confidence 0.83

Every check the pipeline had said yes:

**OK** valid Indian format

**OK** real state code GJ

**OK** real RTO code 03

**OK** confidence 0.83

...and the four digits that identify the vehicle were wrong. 8988 against a true 4879.

6.5 px per character - the recogniser was not reading glyphs, it produced a plausible shape.

Fix: MIN\_ALERTABLE\_PX\_PER\_CHAR = 12. Low-resolution reads are indexed and searchable,

but can never raise an alert.

**Operational Guardrail:** At 6.5 px/char on cam10 (Junagadh), OCR returned valid plate `GJ038988` (confidence 0.83) against true plate `GJ03HR4879`. Rather than ignoring this edge-case, Sentinel implemented `MIN_ALERTABLE_PX_PER_CHAR = 12`: low-resolution reads remain searchable for investigation, but are strictly prohibited from raising automated high-priority alerts. Full transparent limitations are in HLD Section 12.

## Slide 11: Impact on Policing & Strategic Roadmap

### Operational Transformation

Today, an investigator manually contacts multiple departments, scrubs hours of footage, and assembles timelines over several days. With Sentinel, they type a vehicle registration once - every sighting across all connected cameras appears on an interactive map within seconds.

### Strategic Roadmap

1. **Phase 1:** Pilot deployment across 1 district (50-200 cameras) at toll plazas and checkpoints.
2. **Phase 2:** Multi-VMS adapter rollout across urban junctions (Model 3 integration).
3. **Phase 3:** Live VAHAN API credential activation and edge node expansion.
4. **Phase 4:** Statewide edge node deployment across 80,000 cameras (Model 4 scale-out).

## Slide 12: Verification & Reproducibility

| Claim / Benchmark              | Verification Command / URL                                 |
|--------------------------------|------------------------------------------------------------|
| ANPR Throughput Benchmark      | <code>python anpr_worker.py --benchmark</code>             |
| Ground-Truth 6/6 Accuracy Test | <code>python tools/make_sample_feed.py --validate</code>   |
| Grammar & System Test Suite    | <code>python -m pytest tests/ -q</code> (57 tests passing) |
| Ingest & Provenance Endpoint   | <code>GET /api/v1/ingest/status</code>                     |

| Claim / Benchmark        | Verification Command / URL                  |
|--------------------------|---------------------------------------------|
| Audit Trail API          | <code>GET /api/v1/analytics/audit</code>    |
| Interactive OpenAPI Docs | <code>http://localhost:8000/api/docs</code> |